

Grille d'Évaluation - Projet Wordle

Total de l'évaluation : /100 points

1. Modélisation (30 points / 30%)

La capacité à identifier les concepts métier et à les représenter avec des types expressifs.

- **Types et entités du domaine /15**

- Les concepts métier sont représentés par des types dédiés (pas juste des `string` et des `number` partout).
- Les types sont expressifs et reflètent le vocabulaire du jeu (mot, tentative, feedback, état de partie, etc.).
- Le modèle est cohérent : on comprend les règles du jeu en lisant les types.
- Les erreurs métier sont modélisées avec des types dédiés (ex: mot invalide, partie terminée), pas de simples `throw new Error` génériques.

- **Domaine pur et séparation métier/infra /15**

- La logique du jeu ne contient aucune dépendance technique (`console.log` , `fetch` , accès fichiers, etc.).
 - Le code métier est agnostique de l'infrastructure : il ne sait pas comment les mots sont stockés ni comment le jeu est affiché.
 - L'affichage (CLI ou frontend) consomme le modèle — le modèle n'a pas conscience de l'affichage.
-

2. Qualité des tests (20 points / 20%)

Votre suite de tests doit servir de documentation vivante du comportement du jeu.

- **Structure et lisibilité /10**

- Utilisation du canevas **Given / When / Then** (via des blocs visuels ou l'organisation interne du test).
- Les noms des tests construisent des phrases compréhensibles décrivant un comportement métier.
- Pas de "données magiques" : les variables portent des noms explicites (`secretWord` , `playerGuess` , pas juste `"abc"`).

- **Comportement vs. Implémentation /10**

- Les tests vérifient ce que fait le code (valeurs de retour, changements d'état), pas comment il le fait.
 - Si on refactorise la logique interne sans changer le comportement, les tests ne cassent pas.
 - Il n'y a pas d'état global partagé qui fuit entre deux tests.
-

3. Isolation (25 points / 25%)

La capacité à découpler le domaine de ses dépendances et à tester de manière déterministe.

- **Interface et injection de dépendances /10**

- Le dictionnaire est représenté par une interface/un contrat (pas un accès direct à un fichier ou une API).
- La dépendance est injectée dans le jeu (via le constructeur ou un paramètre), pas instanciée en interne.

- **Doublures de test /10**

- Une ou plusieurs doublure de test, notamment du dictionnaire sont utilisées dans les tests.
- Les tests contrôlent le mot secret et la liste de mots valides, garantissant leur déterminisme.
- Les différents types de doublures sont utilisés à bon escient (pas d'abus de mocks génériques).

- **Couverture des edge cases /5**

- La règle des lettres multiples est testée scrupuleusement.
 - Les cas d'erreur sont couverts : mot invalide, mot de mauvaise longueur, jouer après fin de partie.
 - Les cas nominaux sont couverts : victoire, défaite après 6 tentatives, feedback correct.
-

4. Application fonctionnelle (25 points / 25%)

Le projet doit être livré sous forme d'une application jouable.

- **Interface utilisable /15**

- L'application (CLI ou frontend) permet de jouer une partie complète de Wordle.
- Le feedback est affiché de manière compréhensible après chaque tentative.
- Les états de fin de partie (victoire/défaite) sont gérés.

- **README et mise en route /10**

- Le `README.md` explique clairement comment installer les dépendances.
- Les commandes pour lancer les tests et lancer l'application sont documentées.
- Le projet fonctionne en suivant les instructions du README (ça tourne).